

PROJECT REPORT

Campus Course Records Manager (CCRM)

An Object-Oriented Console Application in Core Java

Submitted by

SHASHANK • [25BAI10569]

[CSE-AIML]

Table of Contents

1. Introduction	3
2. Objectives and Scope.....	3
2.1 Objectives.....	3
2.2 Scope	3
3. System Architecture	3
3.1 Class relationships.....	4
4. Class Design	5
4.1 Person — the abstract base	5
4.2 Instructor.....	5
4.3 CourseCode — an immutable value object	5
4.4 Course and the Builder pattern	6
4.5 Grade and Semester — enumerations with behaviour	6
4.6 Enrollment — the associative entity.....	7
4.7 StudentService	7
5. Object-Oriented Principles in Practice	8
6. Design Patterns Applied.....	8
6.1 Builder	8
6.2 Value Object.....	8
6.3 Service Layer	9
7. Java Language Features Used.....	9
8. Representative Usage	10
9. Testing and Known Limitations	10
9.1 Verification approach.....	10
9.2 Limitations.....	10
10. Future Enhancements	11
11. Conclusion.....	11
References.....	11

1. Introduction

Most academic departments still track course registrations and internal marks in spreadsheets that are copied, renamed and mailed around until nobody is sure which version is current. The Campus Course Records Manager, referred to throughout this report as CCRM, was written as a small answer to that problem and, more importantly, as a vehicle for practising object-oriented design in Java rather than merely reading about it.

CCRM is a console-based application that maintains records of students, instructors and courses, links students to courses through enrolment records, and records grades against those enrolments so that academic performance can be computed. The scale is deliberately modest. The intent was never to compete with an institutional ERP but to build something whose every class could be justified on design grounds and explained in a viva.

The project is written entirely in core Java with no external libraries, no build tool and no database. Every construct in the codebase — abstraction, inheritance, polymorphism, encapsulation, enumerated types, the Builder pattern, immutable value objects and a service layer — was chosen because it earns its place, not because a checklist demanded it.

2. Objectives and Scope

2.1 Objectives

- Model the academic domain — people, courses, enrolments and grades — using classes whose responsibilities are clearly separated.
- Demonstrate the four pillars of object-oriented programming through code that genuinely needs them.
- Apply the Builder creational pattern to an object with several optional-looking constructor parameters.
- Use Java enumerations with state and behaviour rather than as bare constant lists.
- Separate domain objects from the logic that manipulates them by introducing a service layer.
- Keep the code readable enough that a reviewer can follow it without a diagram.

2.2 Scope

Within scope: registration of students and instructors, definition of courses with credit values and semester allocation, enrolment of a student in a course, assignment of a letter grade to an enrolment, and retrieval of students by identifier.

Outside scope: persistence to disk or database, a graphical or web interface, authentication, concurrent access, and attendance tracking. These are discussed in Section 10 as candidates for a second iteration.

3. System Architecture

The application follows a two-layer package structure. Domain classes hold state and enforce invariants about themselves; service classes hold collections of domain objects and expose operations over them. No domain class knows that a service exists, which keeps the dependency direction one-way and makes the domain layer independently testable.

Table 3.1 — Package organisation

Package	Type	Responsibility
edu.ccrm.domain	Entity / Value	Person, Student, Instructor, Course, CourseCode, Enrollment
edu.ccrm.domain	Enumeration	Grade, Semester
edu.ccrm.service	Service	StudentService and further services as the system grows

The dependency graph is acyclic. edu.ccrm.service imports from edu.ccrm.domain; the reverse import never occurs. This is a small discipline but it is the difference between a layered application and a folder of classes.

3.1 Class relationships

The relationships in the model are of three kinds. Instructor and Student extend Person, which is generalisation. Course holds a CourseCode and a Semester, which is composition — a course cannot meaningfully exist without them. Enrollment holds references to a Student and a Course without owning either, which is association, and it is what makes Enrollment an associative entity rather than a plain data holder.

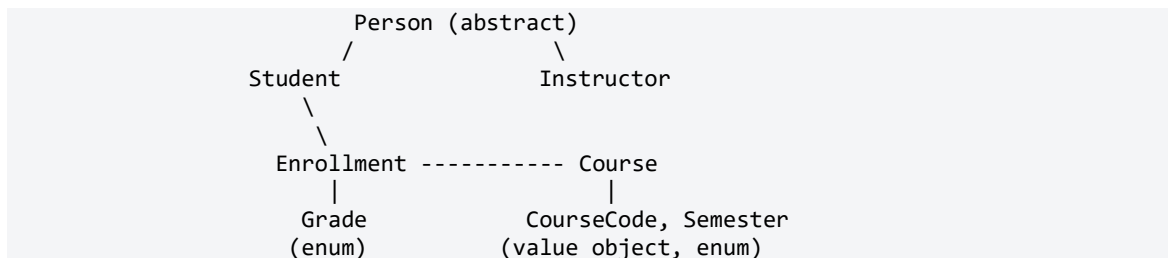


Figure 3.1 — Simplified class relationship diagram

4. Class Design

This section walks through each class in the order a reader would naturally encounter it, explaining not just what the class does but why it is shaped the way it is.

4.1 Person — the abstract base

Person carries the three attributes every human in the system has: an identifier, a full name and an email address. It is declared abstract because there is no such thing as a generic person in a campus records system; every person is either a student or an instructor. Declaring it abstract makes that fact enforceable by the compiler instead of relying on convention.

```
public abstract class Person {
    protected String id;
    protected String fullName;
    protected String email;

    public Person(String id, String fullName, String email) { ... }

    public abstract void printProfile();
}
```

The fields are protected rather than private so that subclasses can read them directly when building their own profile output. The single abstract method, `printProfile()`, is the polymorphic hook: it declares that every concrete person must be able to describe itself, without saying anything about how.

4.2 Instructor

Instructor extends Person and adds a department. Its override of `printProfile()` produces a line specific to teaching staff, drawing on the inherited `fullName` field alongside its own department field.

```
@Override
public void printProfile() {
    System.out.println("Instructor: " + fullName + " | Dept: " + department);
}
```

The `@Override` annotation is not decoration. If the base method signature is ever renamed, the compiler flags this method immediately rather than silently allowing it to become an unrelated method that nothing calls.

4.3 CourseCode — an immutable value object

A course code such as CSE201 could have been stored as a plain `String` field on `Course`. It was wrapped in its own type instead, for a reason worth stating clearly: a `String` parameter can be confused with any other `String` parameter, whereas a `CourseCode` cannot. The class is declared `final` with a single `final` field and no setter, so an instance is immutable from construction and safe to share.

```
public final class CourseCode {
    private final String code;
    public CourseCode(String code) { this.code = code; }
    public String getCode() { return code; }
}
```

This is the Value Object idiom. Its cost here is one extra class; its benefit is a type system that carries meaning.

4.4 Course and the Builder pattern

Course has five attributes: code, title, credits, instructor and semester. A five-argument constructor is legal Java and also a reliable source of bugs, because nothing prevents a caller from transposing two arguments of the same type. The Builder pattern was applied to remove that hazard.

```
Course c = new Course.Builder()
    .setCode("CSE201")
    .setTitle("Data Structures")
    .setCredits(4)
    .setInstructor("Dr. Rao")
    .setSemester(Semester.FALL)
    .build();
```

Three design decisions support this. The Course constructor is private, so Builder is the only route to an instance. Each setter returns this, which is what enables the fluent chain. The static nested Builder mirrors the fields of the enclosing class and assembles them in build().

A pleasant side effect visible in the code is that setCode() accepts a String and performs the wrapping into CourseCode internally, so callers get the safety of the value object without having to think about it.

4.5 Grade and Semester — enumerations with behaviour

Semester is a plain enumeration of SPRING, SUMMER and FALL. Grade is more interesting: each constant carries the grade points associated with it, which turns the enum into a small lookup table that cannot be inconsistent with itself.

```
public enum Grade {
    S(10), A(9), B(8), C(7), D(6), E(5), F(0);

    private int points;
    Grade(int p) { this.points = p; }
    public int getPoints() { return points; }
}
```

Had this been modelled as integer constants or strings, every grade-to-points conversion would have needed a switch statement duplicated wherever it was required, and a typo in any one of them would produce a wrong transcript. Here the mapping is declared once, at the point of definition.

Table 4.1 — Grade scale implemented in the Grade enumeration

Grade	Points	Interpretation
S	10	Outstanding
A	9	Excellent
B	8	Very good
C	7	Good
D	6	Average
E	5	Pass

Grade	Points	Interpretation
F	0	Fail — no credit awarded

4.6 Enrollment — the associative entity

Enrollment is the join between a student and a course. It is created with both references and no grade, reflecting reality: a student enrolls first and is graded later. The grade is set through `setGrade()` once assessment is complete.

```
public int getGradePoints() {
    return (grade == null) ? 0 : grade.getPoints();
}
```

The null guard in `getGradePoints()` is the one piece of defensive code in the class and it matters. Without it, computing an aggregate over a student's enrolments would throw a `NullPointerException` the moment any single course had not yet been graded, which during a live semester is the normal state of affairs rather than an edge case. Returning zero keeps the arithmetic safe, though as noted in Section 9 it also conflates 'ungraded' with 'failed'.

4.7 StudentService

`StudentService` owns the in-memory collection of students and exposes three operations: add, retrieve all, and find by identifier. It is declared against the `List` interface and instantiated as an `ArrayList`, so the concrete collection can be swapped later without touching any calling code.

```
public Student findStudentById(String id) {
    for (Student s : students) {
        if (s.getId().equals(id)) return s;
    }
    return null;
}
```

The lookup is a linear scan, which is $O(n)$. For a class list of a few hundred students that is entirely acceptable, and the alternative — a `HashMap` keyed on identifier — was rejected on the grounds that it would complicate `getAllStudents()` for a performance gain nobody would perceive. It is, however, the first thing to change if the dataset grows.

5. Object-Oriented Principles in Practice

The four pillars are easy to recite and harder to demonstrate. Each is mapped below to the specific place in this codebase where it does real work.

Table 5.1 — Mapping of OOP principles to implementation

Principle	Where it appears	What it achieves
Abstraction	Person declared abstract with abstract printProfile()	Defines the contract for a person without committing to a representation
Inheritance	Instructor and Student extend Person	Common attributes declared once; subclasses add only what is distinct
Polymorphism	printProfile() overridden per subclass	A List<Person> can be iterated and each element prints its own format
Encapsulation	Private fields with controlled accessors throughout	Internal state cannot be corrupted from outside the class

Encapsulation deserves a closer look because it is applied at different strengths in different places. CourseCode is fully immutable. Course exposes a getter for its code but no setters at all, so it is effectively immutable after construction. Enrollment permits exactly one mutation, setGrade(), because that is the only attribute that legitimately changes over the lifetime of an enrolment. In each case the level of access granted matches the level of change the domain actually requires, which is the substance of encapsulation rather than the reflexive habit of generating a getter and setter for every field.

One detail in Course is worth drawing attention to. Its getCode() method returns a String rather than the CourseCode object it holds:

```
public String getCode() {
    return code.getCode();    // unwrap CourseCode into String
}
```

This is a deliberate trade-off. Returning the String keeps calling code simple for display and comparison, at the cost of hiding the value object from consumers. Returning the CourseCode itself would be purer, since the class is immutable and therefore safe to hand out, and would preserve type safety further up the call chain. The current form was kept because callers overwhelmingly want the text.

6. Design Patterns Applied

6.1 Builder

Discussed in Section 4.4. Applied to Course to replace a five-parameter constructor with a self-documenting fluent chain, and to make Builder the sole construction path by keeping the constructor private.

6.2 Value Object

CourseCode wraps a primitive String in a final, immutable type so that a course code is distinguishable from any other string in a method signature.

6.3 Service Layer

StudentService separates collection management from the Student entity. The entity models one student; the service models the registry of all of them. Adding InstructorService and CourseService alongside it is a mechanical extension that requires no change to the domain package.

7. Java Language Features Used

Table 7.1 — Language features and where they occur

Feature	Class	Purpose
Abstract class and method	Person	Enforce a contract at compile time
Static nested class	Course.Builder	Group construction logic with its product
Enum with constructor	Grade	Attach grade points to each constant
final class and field	CourseCode	Guarantee immutability
Generics	List<Student>	Type-safe collections without casting
Enhanced for loop	StudentService	Readable iteration over the student list
Method overriding	toString(), printProfile()	Meaningful output and runtime dispatch
Interface-typed reference	List backed by ArrayList	Decouple code from the concrete collection

The toString() overrides are a minor but practical convenience. Enrollment.toString() composes the student's name, the course's own toString() output and the grade into a single readable line, so printing an enrolment during debugging produces something informative instead of a hash code.

8. Representative Usage

The following sequence shows the classes working together. It is the flow the application performs when a student is registered, a course is defined, an enrolment is created and a grade is recorded.

```
StudentService service = new StudentService();
service.addStudent(new Student("24BCE1001", "A. Sharma",
                               "sharma@campus.edu"));

Course ds = new Course.Builder()
    .setCode("CSE201")
    .setTitle("Data Structures")
    .setCredits(4)
    .setInstructor("Dr. Rao")
    .setSemester(Semester.FALL)
    .build();

Student s = service.findStudentById("24BCE1001");
Enrollment e = new Enrollment(s, ds);
e.setGrade(Grade.A);

System.out.println(e);
System.out.println("Points earned: " + e.getGradePoints());
```

Expected console output:

```
A. Sharma -> CSE201 - Data Structures (4 cr) | Grade: A
Points earned: 9
```

Note that the first line is produced entirely by `toString()` overrides composing with one another: `Enrollment.toString()` calls `Course.toString()`, which in turn unwraps the `CourseCode`. Nothing in the printing code knows the internal structure of any of those objects.

9. Testing and Known Limitations

9.1 Verification approach

Classes were compiled and exercised through a driver class rather than a formal testing framework, since introducing JUnit was outside the stated scope. Each domain class was checked individually — construction, accessor behaviour and `toString()` output — before being combined into the end-to-end flow shown in Section 8. The compiled artefacts confirm that the Builder is emitted as a separate nested class file, `Course$Builder.class`, as expected.

9.2 Limitations

Stating a system's weaknesses honestly is more useful than claiming it has none. The following are known and acknowledged.

- No persistence. All data lives in memory and is lost when the process exits. This is the single largest gap between CCRM and a usable tool.
- No input validation. `CourseCode` accepts any string, including an empty one; credits may be set to a negative number; email addresses are never checked for format.

- equals() and hashCode() are not overridden. CourseCode in particular should implement them, since two codes with identical text are currently unequal, which would cause incorrect behaviour in a HashSet or as a map key.
- Ungraded enrolments return zero points, which is numerically indistinguishable from a grade of F. A nullable Integer or an explicit NOT_GRADED constant would separate the two.
- findStudentById() returns null when no match is found, obliging every caller to remember a null check. Returning Optional<Student> would make the absence explicit.
- Course stores its instructor as a String rather than as an Instructor reference, so the Instructor class is not yet connected to the course model.
- Lookup is a linear scan and duplicate student identifiers are not prevented.

10. Future Enhancements

The limitations above translate directly into a work plan for a second iteration, listed roughly in order of value delivered per unit of effort.

- Persist records to CSV using java.nio.file, then to a relational database through JDBC.
- Replace the String instructor field on Course with an Instructor reference and add InstructorService.
- Introduce a TranscriptService that aggregates a student's enrolments into a weighted GPA using credits and grade points.
- Add validation, either in the Builder's build() method or through a dedicated validator, and raise IllegalArgumentException on violation.
- Override equals() and hashCode() on CourseCode and the entity classes, and index students in a HashMap for constant-time lookup.
- Rewrite search and aggregation using the Streams API, which would reduce findStudentById() to a single filter-and-findFirst expression returning Optional.
- Add JUnit 5 tests covering the Builder, the grade-point mapping and the null-grade path.
- Layer a JavaFX or Spring Boot interface over the existing service layer, which should require no change to the domain package.

11. Conclusion

CCRM does a small job and does it in a way that can be defended line by line. The domain is modelled with classes that correspond to things a registrar would recognise, the relationships between them are explicit, and the design choices — an abstract base rather than a concrete one, a value object rather than a bare String, a Builder rather than a long constructor, an enum carrying points rather than a switch statement — each solve a problem that would otherwise have surfaced later.

The system is not finished, and Section 9 says so plainly. What it does provide is a foundation whose layering is sound: persistence, validation and a graphical interface can all be added above or below the existing code without disturbing the domain classes, which is the practical test of whether a

design was worth the effort. Building it clarified a distinction that is easy to miss when learning Java from examples — that object orientation is not a set of keywords to be used, but a way of deciding where responsibility should sit.

References

1. Bloch, J. Effective Java, 3rd edition. Addison-Wesley, 2018. Item 2 on the Builder pattern and Item 17 on minimising mutability informed the design of Course and CourseCode.
2. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
3. Oracle. The Java Tutorials — Classes and Objects; Enum Types. docs.oracle.com/javase/tutorial
4. Oracle. Java Platform, Standard Edition API Specification — `java.util.List`, `java.util.ArrayList`, `java.util.Optional`.
5. Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002. Reference for the Service Layer and Value Object patterns.